



Universidad Nacional de Rosario

Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Sistemas Operativos I

Trabajo Práctico

Sistema de Gestión de Recursos Distribuido
sobre un Clúster HPC Simulado

División de trabajo

Agente C:	Tomás Demarchi, Santino Mozzi
Estructuras de Datos:	Santino Mozzi, Tomás Demarchi
Agente de Erlang:	Álvaro Cabrera
Implementación, testeo y documentación:	Santino Mozzi, Tomás Demarchi, Álvaro Cabrera

Índice

1. Introducción	3
2. Agente C	3
2.1. Rol de Servidor: gestión de recursos locales, sincronización entre nodos y evasión de <i>deadlocks</i>	4
2.2. Rol de Cliente: solicitud de recursos a nodos remotos	5
2.3. Manejo de desconexiones y errores	5
3. Agente de Erlang	5
3.1. <code>job_generator</code>	5
3.2. <code>coordinator_loop</code>	6
4. Estructuras	6
5. Formatos de Comunicación	7
5.1. Canal Erlang ↔ C local (TCP, 127.0.0.1:4200)	7
5.2. Canal C ↔ C remoto: negociación de recursos (TCP, puerto 4200)	8
5.3. Canal UDP de descubrimiento (<i>broadcast</i> , puerto 12529)	8
5.4. Resumen de transportes	8
6. Aclaraciones	8

1. Introducción

En este proyecto se implementa un sistema de gestión de recursos distribuido basado en un clúster HPC simulado. Para ello, distintos nodos autónomos cooperan entre sí, tanto para administrar sus propios recursos –que serán solicitados por otros nodos– como para crear *workers* que simulen carga de trabajo, solicitando a su vez recursos a los nodos de la red.

Nuestra tarea fue desarrollar uno de estos nodos, el cual debía poder funcionar de forma autónoma, creando trabajos que soliciten recursos y, al mismo tiempo, gestionando los recursos propios del nodo, además de poder conectarse con los demás nodos para solicitar o ceder recursos cuando fuera necesario y posible.

Por ende, la tarea de implementación de nuestro agente se dividió en los siguientes sub-bloques:

- Conectarse correctamente entre las distintas partes del nodo.
- Anunciarse a la red de nodos, comunicando su información de recursos a los demás, y almacenar la información de los nodos conectados.
- Gestionar los recursos del nodo, así como las solicitudes de los trabajos hacia dichos recursos, mediante las estructuras de datos necesarias.
- Crear un *scheduler* de trabajos, los cuales piden recursos según qué nodos los tengan disponibles, simulando carga de trabajo (a través de Erlang).
- Manejar los mensajes entre las distintas partes del nodo, así como los mensajes provenientes de otros nodos.

Estos sub-bloques, a excepción del *scheduler* de Erlang, se agrupan en un componente denominado **Agente C**.

2. Agente C

Como se mencionó anteriormente, este es el bloque principal del programa. Su primera responsabilidad es establecer conexión con el proceso Erlang (Scheduler) y, una vez lograda, poner en funcionamiento un bucle de eventos no bloqueante basado en `epoll`, que actúa como un buzón centralizado de eventos y permite gestionar múltiples conexiones TCP de forma simultánea, sin que la latencia de una petición afecte la operatividad del resto.

La implementación utiliza un *pool* de cuatro hilos de trabajo (*worker threads*) que comparten una misma instancia de `epoll`. Los *sockets* de escucha (conexión con Erlang y conexión con nodos remotos) se registran con la bandera `EPOLLEXCLUSIVE`, de modo que ante una nueva conexión entrante solo un hilo sea despertado, evitando el problema de *thundering herd*. Las conexiones ya establecidas, en cambio, se registran con `EPOLLONESHOT`, garantizando que un mismo *socket* sea procesado por un único hilo por ronda de eventos, sin necesidad de un *mutex* adicional por conexión.

Además del tráfico TCP, el agente gestiona dos temporizadores (`timerfd`) integrados al mismo `epoll`:

- Uno dispara cada 5 segundos el envío de un mensaje **ANNOUNCE** por *broadcast* UDP, informando los recursos libres del nodo.

- Otro dispara, también cada 5 segundos, la revisión de *timeouts*, tanto de los nodos conocidos (vencimiento del ANNOUNCE) como de los *jobs* locales en espera de un recurso.

Gracias a este mecanismo de descubrimiento por *broadcast*, la topología de la red se mantiene dinámica: cada agente arma y actualiza automáticamente su tabla de nodos vecinos, sin necesidad de una configuración estática de direcciones.

Una vez en funcionamiento el bucle de eventos, el agente asume simultáneamente dos roles: **servidor**, para atender las solicitudes de recursos que le llegan de otros nodos, y **cliente**, para solicitar recursos a nodos remotos en nombre de los *jobs* que le asigna el Scheduler.

2.1. Rol de Servidor: gestión de recursos locales, sincronización entre nodos y evasión de *deadlocks*

En cuanto a la administración de recursos, cada agente simula poseer una cantidad limitada de CPU, memoria y GPU, los cuales son otorgados a las solicitudes de *jobs* provenientes de los distintos nodos de la red. Cada tipo de recurso está protegido por un *lock* (*mutexQueue*) propio, de forma de evitar condiciones de carrera al momento de consultar o modificar la cantidad disponible.

Si al momento de solicitarse un recurso este no se encuentra disponible, el agente no rechaza la petición inmediatamente: la encola en una estructura FIFO, donde el *job* queda a la espera de que el recurso sea liberado y otorgado. Esta cola no tiene un límite fijo de tamaño, por lo que puede almacenar tantos trabajos como sea necesario.

Respecto a la prevención de *deadlocks*, la solución se apoya en dos frentes, cada uno atacando una condición distinta del modelo de Koffman:

1. **Ruptura de la espera circular (*circular wait*):** se establece un orden fijo y global para solicitar los recursos: CPU → MEM → GPU. De esta manera, si dos nodos necesitan los mismos recursos entre sí, ambos los piden siempre en el mismo orden, por lo que aquel que logre reservar primero el recurso inicial de la secuencia podrá continuar, mientras el otro queda a la espera de que se libere. Esto rompe estructuralmente la posibilidad de una espera circular. Cabe aclarar que esta solución es válida únicamente si **todos** los agentes que solicitan recursos respetan dicho orden; si existiera un Scheduler externo que pidiera recursos en una secuencia distinta, la espera circular volvería a ser posible. Por este motivo se incorpora una segunda solución complementaria.
2. **Ataque a la No-Apropiación (*no-preemption*):** basada en una lógica de *timeouts*, si un nodo permanece esperando un recurso durante un tiempo determinado sin conseguirlo, es retirado de la cola de espera y se le informa al Scheduler de Erlang que ese *job* sufrió un **TIMEOUT**, para que, transcurrido un intervalo, se relance el mismo *job* solicitando nuevamente los mismos recursos. La lógica detrás de esta decisión es que, si un recurso permanece bloqueado en espera durante demasiado tiempo, lo más probable es que se trate de un *deadlock*; al retirar de la cola al *job* que lleva más tiempo esperando, se libera artificialmente el bloqueo y se permite que el resto de los nodos trabados puedan avanzar.

La combinación de ambos mecanismos permite no solo prevenir *deadlocks* de forma determinística dentro del propio Scheduler (gracias al punto 1), sino también actuar como salvaguarda ante escenarios donde el orden de solicitud de recursos no sea respetado por un agente externo (gracias al punto 2).

De esta manera, la gestión de recursos del servidor resulta correcta y evita bloqueos indefinidos del sistema.

2.2. Rol de Cliente: solicitud de recursos a nodos remotos

Cuando el Scheduler de Erlang envía un **JOB_REQUEST**, el agente C asume el rol de cliente y se encarga de conseguir, uno a uno, cada uno de los recursos que el *job* necesita, respetando siempre el orden CPU → MEM → GPU definido para evitar la espera circular.

Para cada recurso pendiente, el agente abre una conexión TCP no bloqueante hacia el nodo proveedor correspondiente y la registra en **epoll** en modo **EPOLLOUT**. Recién cuando el sistema operativo notifica que el *socket* está listo para escribir, se envía efectivamente el mensaje **RESERVE** <job_id> <recurso> <cantidad>; a partir de ese momento, la conexión se reconfigura a modo **EPOLLIN** para esperar la respuesta del nodo remoto.

Si la respuesta es **GRANTED**, el recurso se marca como obtenido, se reinicia el contador de tiempo del *job* (de forma que el *timeout* se mida por recurso y no por la vida completa del *job*) y se solicita automáticamente el siguiente recurso pendiente de la lista. La conexión utilizada para obtener ese recurso se mantiene abierta –pero se retira del **epoll** y de la tabla de *sockets* activos– ya que su único propósito remanente es enviar, en el futuro, el **RELEASE** correspondiente cuando el *job* finalice.

Cuando ya no quedan recursos pendientes (**next_req == NULL**), el *job* se considera completamente otorgado y el agente notifica a Erlang mediante **JOB_GRANTED**. En cambio, si algún nodo responde con **DENIED** o con cualquier mensaje no reconocido, el *job* se interpreta como rechazado: se liberan los recursos ya obtenidos en otros nodos (evitando dejar recursos reservados sin uso) y se informa a Erlang mediante **JOB_DENIED**.

2.3. Manejo de desconexiones y errores

Finalmente, el agente contempla la posibilidad de que un nodo remoto se desconecte de forma abrupta, ya sea porque terminó su ejecución o por un error de red. Ante un cierre de conexión, se liberan los recursos que ese nodo tuviera reservados a nuestro favor y se limpia toda referencia al *socket* cerrado. De manera similar, si es el proceso Erlang el que se desconecta, el agente elimina su *file descriptor* de **epoll** y descarta la referencia global (**erlangfd**), quedando a la espera de una futura reconexión. Este manejo evita que *sockets* muertos permanezcan indefinidamente registrados en el sistema o que el agente intente operar sobre conexiones inválidas.

3. Agente de Erlang

La lógica del **scheduler_erlang** se separa principalmente en dos bloques concurrentes. En un primer bloque se ejecuta un proceso que se encarga de generar nuevos trabajos (*jobs*) a intervalos regulares de tiempo. En el segundo bloque, un proceso denominado **coordinator_loop** se mantiene a la escucha constante, gestionando el ciclo de vida y el estado de la comunicación mediante el patrón de paso de mensajes.

3.1. job_generator

Es el proceso responsable de instanciar los trabajos. Se estableció que el ID del primer trabajo inicie en 1001 y se incremente linealmente con cada nueva creación. Esta decisión de diseño garantiza la unicidad de los identificadores de manera eficiente, evitando la colisión de IDs en la tabla *hash* del Agente C sin requerir algoritmos de generación complejos. Al iniciar la creación de un trabajo, el generador solicita al Agente C la lista actualizada de nodos en la red, selecciona aleatoriamente los recursos requeridos y ensambla el mensaje de solicitud

(`JOB_REQUEST`) para enviarlo a la capa de red.

3.2. coordinator_loop

Este proceso central actúa como el único puente de comunicación entre los trabajos de Erlang y el *socket* TCP del Agente C, centralizando el tráfico y evitando condiciones de carrera sobre el *socket*. Reacciona a cuatro tipos de eventos mediante *pattern matching*:

i. Inicialización y finalización de trabajos.

Gestiona el registro de los trabajos creados en un `Map` (diccionario) interno y notifica al Agente C. Cuando un trabajo notifica su finalización, el coordinador lo elimina del `Map` y envía la señal de liberación de recursos (`JOB_RELEASE`) a la capa de C.

ii. Estado de las peticiones.

Atrapa los mensajes asíncronos del Agente C (`JOB_GRANTED`, `JOB_DENIED` o `JOB_TIMEOUT`). Independientemente de la señal, registra el evento en el archivo de *log* del sistema y enruta el mensaje al proceso del trabajo correspondiente utilizando el ID almacenado en el `Map`.

iii. Topología de la red (`GET_NODES`).

Cuando el generador necesita crear un trabajo, el coordinador envía la señal `GET_NODES` al Agente C para obtener una visión global del clúster. Al recibir la cadena de nodos, el coordinador se la pasa al generador para que este seleccione de qué nodos remotos se solicitarán los recursos físicos.

iv. Manejo de errores TCP.

Ante la desconexión abrupta o error crítico del *socket* local conectado al Agente C, el coordinador itera sobre el `Map` activo y envía una señal de cancelación inmediata a todos los trabajos en curso, garantizando que ningún proceso quede en estado de inanición o bloqueo permanente.

4. Estructuras

Como ya se mencionó previamente, el Agente C cumple dos roles –cliente y servidor– para los cuales utiliza estructuras de datos distintas. Las estructuras principales son cinco:

`table_nodes`.

Tabla *hash* que almacena los nodos con los que nos conectamos (nótese que nuestro agente se reconoce a sí mismo como un nodo al cual se conecta). Es útil a la hora de buscar información de los recursos de otros nodos, su puerto e IP, así como para desconectar nodos en caso de inactividad. Esta estructura es utilizada principalmente por la parte de servidor del Agente C, al momento de dar información acerca de nodos cercanos y sus recursos. Por último, cada casilla de nodo en esta estructura cuenta con un campo `time_t` para determinar cuándo un nodo conectado ya se encuentra en estado inactivo.

`table_nodejobs`.

Tabla que almacena los *jobs* recibidos por parte de otros nodos, cuyo método de búsqueda *hashea* a través de una combinación del ID, el puerto y la IP del nodo. Esta tabla también es utilizada por la parte de servidor para determinar qué recurso está en espera de ser otorgado, a qué cola hay que encolarlo, entre otras cosas.

table_ourjobs.

Al igual que `table_localjobs`, almacena trabajos locales, pero esta tabla los hasha por el *file descriptor* al cual se conectaron al momento de pedir los recursos, lo que permite un acceso rápido a la hora de buscar un trabajo por el *socket* saliente durante la cadena de reservas.

table_localjobs.

Tabla que busca, mediante un *hash*, los trabajos locales (enviados por nuestro Erlang) por su ID. Se utiliza principalmente para tener acceso rápido a los *jobs* al momento de tener que enviar un **GRANTED/DENIED**. Guarda en cada casilla el *job* junto con la lista de recursos que tiene pendientes u otorgados.

resource_queue.

Cola de tipo FIFO, utilizada una por cada tipo de recurso a gestionar (CPU, memoria, GPU), con el fin de encolar aquellos trabajos que estén en espera de los recursos de nuestro agente local. Cada vez que se recibe una petición de reserva de un *job*, en caso de que la cantidad disponible no sea suficiente, se encola el *job* en la cola correspondiente. Para su correcto funcionamiento, se utiliza un *mutex* con el fin de evitar condiciones de carrera.

5. Formatos de Comunicación

El sistema está compuesto por tres tipos de entidades que se comunican entre sí:

1. El proceso Erlang (*scheduler*): genera *jobs* y habla con su Agente C local.
2. El Agente C local (*agente.c*): intermediario entre Erlang y la red de nodos.
3. Los Agentes C remotos (otros nodos): negocian recursos entre sí.

Todos los mensajes son texto plano en ASCII, terminados en `'\n'` (un mensaje por línea), lo que permite utilizarlos tanto sobre TCP en modo `{packet, line}` (lado Erlang) como con lectura línea a línea manual (lado C). Todos los formatos comparten además la misma filosofía: un comando en mayúsculas seguido de argumentos separados por espacio, y el salto de línea como delimitador de mensaje.

5.1. Canal Erlang ↔ C local (TCP, 127.0.0.1:4200)

Erlang → C

JOB_REQUEST <job_id> <@ip:recurso:cantidad> [...]

Pide iniciar un *job* con una lista de recursos a reservar, uno por cada nodo destino.

JOB_RELEASE <job_id>

Indica que el *job* terminó y hay que liberar sus recursos.

JOB_STATUS <job_id>

Pregunta el estado actual de un *job*.

GET_NODES

Pide la lista de nodos conocidos y sus recursos libres.

C → Erlang

JOB_GRANTED / **JOB_DENIED** / **JOB_TIMEOUT** <job_id>

Respuestas al ciclo de vida de un *job*.

WAITING <job_id>

El *job* sigue esperando (respuesta a **JOB_STATUS**).

NODES <lista_de_nodos>

Respuesta a **GET_NODES**.

5.2. Canal C ↔ C remoto: negociación de recursos (TCP, puerto 4200)

RESERVE <job_id> <recurso> <cantidad>

Un nodo pide reservar un recurso (cpu/mem/gpu) de otro nodo.

RELEASE <job_id>

Libera un recurso previamente reservado en ese nodo.

GRANTED <job_id>

El nodo proveedor concede el recurso pedido.

Nota: cualquier otro mensaje, o la ausencia de **GRANTED**, se interpreta como denegación o rechazo del pedido. Los recursos se solicitan siempre en el orden fijo CPU → MEM → GPU para evitar la espera circular (*circular wait*) entre nodos.

5.3. Canal UDP de descubrimiento (*broadcast*, puerto 12529)

ANNOUNCE <puerto> cpu:<n> mem:<n> gpu:<n>

Cada nodo emite periódicamente (cada 5 s) sus recursos libres por *broadcast* UDP. La IP del emisor no viaja en el mensaje: se obtiene de `recvfrom()` en el nodo receptor. Con esto, cada nodo arma y actualiza su tabla de nodos conocidos (`table_nodes`).

5.4. Resumen de transportes

Canal	Transporte
Erlang ↔ C local	TCP, <code>localhost</code> , puerto 4200, <i>framing</i> por línea
C ↔ C (negociación)	TCP, puerto 4200, <i>framing</i> por línea
Descubrimiento de nodos	UDP <i>broadcast</i> , puerto 12529

6. Aclaraciones

Al ejecutar el agente bajo **Valgrind** se reportan pérdidas de memoria (*memory leaks*) al finalizar la ejecución. Estas pérdidas no se deben a errores de gestión de memoria dentro de la lógica del programa, sino a que el proceso es interrumpido de forma abrupta (por ejemplo, mediante **Ctrl+C** o al cortar manualmente la ejecución) en lugar de finalizar a través de un camino de salida controlado.

Dado que el agente está diseñado para ejecutarse de forma indefinida —como corresponde a

un nodo de un clúster real, que permanece activo escuchando eventos–, no existe un flujo de finalización ordenado que libere explícitamente cada estructura reservada dinámicamente (tablas *hash*, colas de recursos, nodos, *jobs* pendientes, etc.). Al cortar el proceso en medio de su ejecución, toda la memoria que en ese momento estuviera en uso queda reportada por Valgrind como no liberada, aun cuando el programa nunca pierde la referencia a dicha memoria mientras está corriendo ni la reserva de forma repetida o innecesaria.

En otras palabras, las pérdidas reportadas son *consistentes con el diseño* del agente y no evidencian una fuga de memoria real (crecimiento no acotado de memoria durante la ejecución), sino simplemente memoria todavía en uso al momento del corte forzado del programa.